

Ejercicio de refuerzo - Semana 3

CPU y GPU con tensores y entrenamiento en PyTorch

Modalidad	Tiempo sugerido	Entrega
Individual	60-90 minutos	Notebook ejecutado y resultados

Versión adaptada para el solver (Taller 03 v2, MMIA 6013). El contenido técnico, los datos, las constantes y los resultados esperados son los del enunciado original del curso. Solo cambia la logística de entrega: no se usa Google Colab ni D2L. Se ejecuta en el equipo local, en la GPU CUDA si existe y en CPU si no. La configuración, las tareas A-D y las preguntas P1-P6, que en el original vivían en un notebook inicial, se transcriben aquí para que el enunciado sea autosuficiente.

Objetivo

Aplicar operaciones tensoriales, autodiferenciación y un ciclo básico de entrenamiento en PyTorch. Comparar CPU y GPU mediante mediciones controladas y explicar cuándo los resultados permiten observar una ventaja de tiempo.

Escenario

Se ejecutarán los mismos productos matriciales y un entrenamiento afín en los dispositivos disponibles. Los datos son sintéticos y se generan en el notebook con una semilla declarada; no representan una población real. No hay descarga de datasets.

Configuración (obligatoria, no modificar)

```
torch.set_default_dtype(torch.float32)
torch.set_float32_matmul_precision("highest")
HAY_GPU = torch.cuda.is_available()
DISPOSITIVOS = [cpu] + ([cuda:0] si HAY_GPU)
SEMILLA = 20260905      TAMANOS = (64, 512, 2048)
PASOS = 40              LR = 0.1
REPETICIONES_MM = 7     REPETICIONES_TRAIN = 3
ATOL = RTOL = 1e-4
```

Registrar el entorno en una tabla: versiones de Python y PyTorch, plataforma, CPU, hilos, runtime CUDA, nombre y memoria de la GPU (o None), dtype, precisión, semilla y la procedencia «mediciones propias de esta sesión».

Sin GPU

Continúe la ruta CPU y declare la limitación. **No invente tiempos ni razones CPU/GPU.** Si no existe una medición GPU real de referencia, explique qué comparación quedó pendiente. La falta de hardware no se penaliza.

Tarea 1 (A) - Identificar el dispositivo

Implementar `mover(tensor, dispositivo)`, que retorna el tensor trasladado con `.to()`. Para cada dispositivo, comprobar con `[[1,2],[3,4]]` que el resultado es un tensor en ese dispositivo, con el mismo dtype, la misma forma y los mismos valores. Responder P1 **antes de medir**.

Tarea 2 (B) - Comparar productos matriciales

Implementar `producto(A, B)` con el operador `@` (no `*`). Comprobación: `[[1,2],[3,4]] @ [[2],[1]] = [[4],[10]]`. Responder P2.

Protocolo de medición para n en TAMANOS: generar en CPU $A = \text{randn}(n, n) / \sqrt{n}$ y $B = \text{randn}(n, n)$ con un generador sembrado con SEMILLA + n ; copiar los mismos datos a cada dispositivo fuera del cronómetro. Dos bloques de calentamiento, luego 7 mediciones; cada medición agrupa 20 productos si $n = 64$, 5 si $n = 512$ y 1 si $n = 2048$, y se divide por esa cantidad. Sincronizar la GPU antes de iniciar y después de terminar cada medición. Reportar

mediana, mínimo y máximo en ms, la concordancia con el resultado CPU (allclose con ATOL y RTOL) y, si hay GPU, la razón CPU/GPU = tiempo CPU ÷ tiempo GPU. Medir además, en otra tabla, la operación CPU → GPU → producto → CPU con transferencias incluidas. Graficar los tiempos en escala logarítmica. Responder P3. Conservar resultados desfavorables o inesperados.

Tarea 3 (C) - Observar la retropropagación

Modelo: predicción = $X @ W + b$. Pérdida: suma de errores cuadrados por fila, promedio sobre filas, dividido por 2 (escalar, shape ()). Implementar `calcular_gradientes(loss)`, que llama a `loss.backward()`. Caso pequeño, en cada dispositivo:

```
X = [[1,2],[3,4]]  Y = [[1],[2]]  W = zeros(2,1)  b = zeros(1)
Esperado tras backward: W.grad = [[-3.5],[-5.0]]  b.grad = [-1.5]
W no cambia con backward.
```

Si hay GPU, comparar los gradientes CPU y GPU con la tolerancia indicada. Responder P4 distinguiendo el valor del parámetro de su gradiente.

Tarea 4 (D) - Completar el entrenamiento

Implementar `paso(X, Y, W, b, optimizador)` en este orden: reiniciar gradientes (`zero_grad`), `forward` y pérdida, `calcular_gradientes`, `step`; retornar `loss.detach()`. Optimizador: `torch.optim.SGD`, `lr = 0.1`, sin momentum (descenso de gradiente full-batch). Comprobación con el caso pequeño, cargando antes gradientes previos de 100 para verificar el reinicio:

```
Tras el paso 1: W = [[0.35],[0.5]]  b = [0.15]
Tras el paso 2: W = [[0.145],[0.21]]
```

Guardar una traza del paso 1 (`W[0,0]` antes, gradiente, `lr`, `W[0,0]` después, `b` después) y usarla en P5.

Experimento de tiempos: con un generador sembrado con SEMILLA, $X = \text{randn}(2048, 256)$, $W_{\text{obj}} = \text{randn}(256, 128) / \sqrt{256}$, $b_{\text{obj}} = \text{randn}(128) \cdot 0.1$, $Y = X @ W_{\text{obj}} + b_{\text{obj}}$. Parámetros iniciales en cero. 40 pasos completos, 3 repeticiones por dispositivo tras un calentamiento; reiniciar parámetros y optimizador antes de cada repetición, no antes de cada paso. Reportar la mediana del tiempo de 40 pasos, la pérdida inicial y la final (evaluada con `torch.no_grad()`), y la curva de pérdida de la última repetición. Sin impresiones ni transferencias dentro del bucle. Responder P6.

Preguntas

- P1 Antes de medir, prediga si la GPU ganará en todos los tamaños y explique su idea. Luego registre el hardware disponible y cite shape, dtype y device de un tensor. ¿Activar un acelerador basta para trasladar el tensor? Conserve la predicción inicial.
- P2 En el ejemplo A de forma (2,2) y B de forma (2,1), explique la forma del producto y calcule su primera entrada. Para matrices (n,n), ¿qué dimensión tiene la salida? Distinga @ de *.
- P3 Compare dos tamaños: cite tiempos, razones CPU/GPU y contraste su predicción P1. Para un tamaño, compare GPU con y sin transferencias. ¿Qué comparación responde a repetir cálculos con datos ya en GPU? Si no tiene GPU ni referencia real, reporte dos tiempos CPU y explique qué conclusión queda pendiente; no invente razones.
- P4 En el caso pequeño, registre pérdida, `W[0,0]`, su gradiente y su dispositivo después de `backward`. ¿Cambió W? Explique qué información aporta el signo del gradiente. Si hay GPU, cite la comprobación de cercanía; si no, indique que no se ejecutó esa comparación.
- P5 Use la traza para completar numéricamente $W_{\text{nuevo}} = W_{\text{anterior}} - 0.1 \times \text{gradiente}$. ¿Qué hace `zero_grad`, qué hace `backward` y qué hace `step`? ¿Por qué el siguiente paso necesita un nuevo `forward`?
- P6 Compare tiempo, pérdida inicial y final del entrenamiento, y la cercanía de parámetros si hay GPU. ¿Más rápido significa aprender mejor? Redacte una conclusión de 80-120 palabras con dos evidencias y una limitación. Sin GPU ni referencia, limite la conclusión a CPU y declare lo no observado.

Entregable

- Un notebook Ejercicio_3.ipynb **ejecutado** de principio a fin sin errores, con las tablas y figuras como salida de celda y las respuestas P1-P6 en celdas Markdown.
- Una carpeta resultados_ejercicio3/ con resultados.json (entorno, estado de las tareas, ruta CPU o CPU y GPU, y todos los resultados), los CSV productos, transferencias, gradientes, traza_paso, entrenamiento y las figuras tiempos_producto.png y perdida_entrenamiento.png.
- Toda cifra de las respuestas debe provenir de una ejecución. No se exige una GPU concreta, una aceleración mínima ni igualdad bit a bit entre dispositivos.

Criterios de evaluación

- Implementación A-D: 40 puntos (transferencia 8, producto 8, backward 8, ciclo 16).
- Interpretación P1-P6: 40 puntos (P1 y P2: 5 cada una; P3: 10; P4 y P5: 6 cada una; P6: 8).
- Evidencia y entrega: 20 puntos (entorno y procedencia 6; resultados conservados 8; notebook ejecutable y archivos entregados 6).